

Lab Session 6: Log Analysis & Incident Response

Date: 11 July 2026

Duration: 45 minutes

Scenario:

The SOC received an alert that a web server was acting slowly between 02:00 and 04:00 AM. The task was to analyse the logs to determine if an attack occurred and what actions were taken by the threat actor. Auth logs were examined for SSH brute force activity, and Apache access logs were examined for web-based attack patterns including SQL injection and directory traversal.

Objective:

Analyse Linux system logs and Apache web server logs to detect attacks, brute-force attempts, suspicious logins, and error spikes using command-line tools. The goal was to build command pipelines to extract actionable intelligence from raw log data — a core skill in SOC analysis and incident response.

Environment:

Tools Used: bash, grep, awk, cut, wc, sort, uniq, tail, nano

Log Files: /var/log/auth.log(simulated)+/var/log/apache2/access.log (simulated)

Output: incident_report_YYYYMMDD.txt - timestamped incident summary

Virtual Machines Running: Kali Linux (live USB) - no target VM required

Pre LAB issues

Before beginning my exploit I had to setup my log files for the simulation:

```
sudo bash -c 'cat > /var/log/auth.log << EOF
```

```
Jul 9 02:01:23 server sshd[1234]: Failed password for root from 45.33.32.156  
port 22 ssh2
```

```
Jul 9 02:01:25 server sshd[1234]: Failed password for root from 45.33.32.156  
port 22 ssh2
```

```
Jul 9 02:01:27 server sshd[1234]: Failed password for root from 45.33.32.156  
port 22 ssh2
```

```
Jul 9 02:03:15 server sshd[1235]: Failed password for admin from  
198.54.117.197 port 22 ssh2
```

```
Jul 9 02:03:17 server sshd[1235]: Failed password for admin from  
198.54.117.197 port 22 ssh2
```

```
Jul 9 02:05:44 server sshd[1236]: Accepted password for msfadmin from  
192.168.56.1 port 22 ssh2
```

```
Jul 9 03:21:02 server sshd[1237]: Failed password for root from 203.0.113.42  
port 22 ssh2
```

```
Jul 9 03:21:04 server sshd[1237]: Accepted password for root from 45.33.32.156  
port 22 ssh2
```

```
EOF'
```

Firs Attempt,SSH Brute Force Detection:

First I checked all SSH login Attempts:

```
grep 'Failed password' /var/log/auth.log
```

```
grep 'Failed password' /var/log/auth.log | wc -l
```

I detected 6 failed SSH login attempts between 02:01 and 03:21 am

Next: Searching for Top Attacking IPs

```
grep 'Failed password' /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c |  
sort -rn | head -20
```

Results:

3 45.33.32.156 # Primary attacker - most aggressive

2 198.54.117.197 # Secondary attacker

1 203.0.113.42 # Third attacker

On that I found 3 distinct source IPs conducting SSH brute force, 45.33.32.156 was the most aggressive with 3 attempts.

Successful Logins After Brute Force:

For the next step I checked successful logins:

```
grep 'Accepted' /var/log/auth.log
```

45.33.32.156 successfully authenticated as ROOT at 03:21:04 — CONFIRMED BREACH. Same IP responsible for 3 failed attempts earlier.

Logins at Suspicious Hours:

```
awk '/02:[0-9][0-9]:[0-9][0-9]|03:[0-9][0-9]:[0-9][0-9]/'
```

```
/var/log/auth.log | grep 'Accepted'
```

This confirmed the msfadmin login at 02:05 during the suspicious window. The root login at 03:21 from the attacker IP was also identified in the previous grep output.

Web Server Log Analysis (Part B)

Recent Web Requests

tail -50 /var/log/apache2/access.log

Displayed all 8 web requests in the simulated access log, showing the full Apache log format: IP, timestamp, HTTP method, URL, status code, and response size.

Top Source IPs in Web Logs

awk '{print \$1}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head -10

4 45.33.32.156 # Same IP from SSH brute force!

2 198.54.117.197 # Same IP from SSH brute force!

1 192.168.56.1

1 192.168.1.100

45.33.32.156 and 198.54.117.197 are hitting BOTH SSH and the web server signs of a coordinated multi-vector attack

404 Errors (Scanning Behaviour)

grep '404' /var/log/apache2/access.log | wc -l

grep '404' /var/log/apache2/access.log | awk '{print \$7}' | sort | uniq -c | sort -rn | head -20

3 total 404 errors:

1 /wp-admin # WordPress admin probe

1 ../../../../etc/passwd # Directory traversal attempt

1 /admin # Admin panel probe

Directory traversal attempt detected — attacker tried to read /etc/passwd via web

SQL Injection Detection

grep -i 'union\\|select\\|insert\\|drop' /var/log/apache2/access.log

SQL injection detected: GET /index.php?id=1 UNION SELECT 1,2,3-- from 198.54.117.197 — returned HTTP 200 (successful response)

Directory Traversal Detection

grep '\\.' /var/log/apache2/access.log

Directory traversal: GET ../../../../etc/passwd from 198.54.117.197 — attacker

Traffic Volume by Hour

awk '{print \$4}' /var/log/apache2/access.log | cut -d: -f2 | sort | uniq -c | sort -rn

5 requests at 02:xx # Peak attack time

2 requests at 03:xx # Continued activity

1 request at 04:xx # Tapering off

Traffic spike confirmed during 02:00–04:00 AM window — matches SOC alert timeframe exactly

Incident Response Summary Script (Part C)

An automated bash script was built to generate a quick incident summary from the log files — the kind of script a SOC analyst would run at the start of an investigation to get an immediate picture of what happened:

#!/bin/bash

echo '=== INCIDENT RESPONSE SUMMARY ==='

```
echo "Date: $(date)"
```

```
echo ''
```

```
echo '-- FAILED SSH ATTEMPTS --'
```

```
grep 'Failed password' /var/log/auth.log | wc -l
```

```
echo '-- TOP ATTACKING IPs --'
```

```
grep 'Failed password' /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c |  
sort -rn | head -5
```

```
echo '-- SUCCESSFUL LOGINS --'
```

```
grep 'Accepted' /var/log/auth.log | tail -10
```

```
echo '-- WEB SERVER 404s --'
```

```
grep '404' /var/log/apache2/access.log | wc -l
```

```
echo '=== END OF SUMMARY ==='
```

The script was made executable and run, saving output to a timestamped report file:

```
chmod +x incident_summary.sh
```

```
./incident_summary.sh | tee incident_report_$(date +%Y%m%d).txt
```

Note: the first run of the script failed with 'unexpected EOF while looking for matching' — caused by a missing closing bracket on the \$(date) command. The script was corrected in nano and ran successfully on the second attempt.

Analyst Workflow Summary:

- ❖ DETECT — used grep/awk to find anomalies in auth.log and apache access.log
- ❖ COUNT — used wc -l and uniq -c to measure the scale of attacks
- ❖ IDENTIFY — extracted attacker IPs, targeted paths, and timestamps
- ❖ CORRELATE — cross-referenced auth.log and access.log to confirm the same IPs conducting a multi-vector attack
- ❖ REPORT — built an automated bash script to generate a timestamped incident summary
- ❖ RESPOND — next step would be blocking top attacking IPs with iptables or fail2ban

Findings:

- ❖ Powerful security intelligence can be extracted from raw log files using nothing but built-in Linux command-line tools – no expensive SIEM required.
- ❖ The primary threat actor (45.33.32.156) conducted SSH brute force attacks and successfully authenticated as root.
- ❖ A secondary attacker (198.54.117.197) performed SQL injection and directory traversal attacks against the web server.
- ❖ Correlating auth.log and access.log by IP revealed a coordinated multi-vector attack rather than two unrelated incidents.
- ❖ grep, awk, cut, and sort are enough to triage an incident and identify the key facts needed to respond.

Lessons Learned

- ❖ Always cross-reference log sources by IP – the same attacker often appears across multiple services.
- ❖ Build pipelines incrementally – grep first to isolate the pattern, then awk/cut to extract fields, then sort | uniq -c to count.
- ❖ Test automation scripts on a small scale before trusting the output – a single missing bracket broke the first run of the incident summary script.
- ❖ Timestamps are as important as IPs – correlating the attack window (02:00-04:00 AM) against the original SOC alert confirmed the investigation was on the right track

Note to Self:

Start every log investigation with grep 'Failed'/'Accepted' and a 404 count — those two checks alone flag most incidents fast. Always correlate IPs across log sources before writing conclusions, and test bash scripts on a small case before relying on them during a real incident.